SkillMerge 심화 해설서 — 개별 개선은 왜 조합의 성공을 보장하지 않는가

작성 기준: 2026-10-12. 대응 원고: 첫 주 연구 설계용 4페이지 영문 paper.json. 이 해설서는 원고의 정의·수식·평가 설계를 풀어 쓰는 문서다. 실제 LLM의 스킬 조합 성능, 자연 발생 빈도, 통합 정책의 우월성은 모두 NR이다. 최종 모델·표본 수·개선 문턱·실제 비용 한도·사전등록은 아직 고정되지 않았다. 결정론적 모형과 가상 수치는 각각 표시하며, 준비한 데이터나 소프트웨어 검사 수를 성능 실험 수로 바꾸지 않는다.

1. 네 페이지를 읽는 지도와 근거의 종류

이 연구의 짧은 질문은 "같은 조상에서 각각 좋아진 두 스킬 업데이트를 합쳐도 좋아지는가?"이다. 그러나 '좋아졌다', '합쳤다', '실패했다'라는 말은 모두 평가 규칙을 필요로 한다. 파일이 정리되었다거나 모델이 좋은 수정이라고 평가한 것은 개별 개선의 증거가 아니다. 실제 과제에서 별도로 확인한 성공확률이 높아져야 개선이라고 부를 수 있다. 두 파일을 한 폴더에 넣었다는 사실도 두 파일의 내용이 실제 실행에 사용되었다는 증거와 다르다.

영문 원고	중심 질문	이 해설서의 대응 절
1쪽	연구 대상과 주 추정량은 무엇인가	1–3, 7절
2쪽	계약 반례·보존 조건·Q/C/S 분리는 무엇인가	4–8절
3쪽	어떤 과제·대조군·표본으로 평가하는가	9–14, 17절
4쪽	정책 비교·반증·첫 주의 완료 조건은 무엇인가	15–20절

문서에서 말하는 직접 준비 근거는 로컬 파일을 만들어 입력·라벨·분할을 검사한 기록이다. 예를 들어 SciFact를 변형한 531개 입력은 실제로 준비되어 있다. 하지만 그 입력에 LLM이 응답한 성능 결과는 없다. 저자의 보고는 외부 논문이 주장하는 내용이다. 공식 초록이나 방법 절을 읽었다는 범위를 적고, 우리 프로젝트가 그 결과를 재현한 것으로 쓰지 않는다. 설계 제안은 앞으로 실행할 순서와 비교 방법이고, 형식 모형의 계산은 우리가 정의한 함수에 값을 넣어 얻는 논리적 결과다.

NR은 '측정되지 않았다'는 뜻이다. 성공률이 0이라는 뜻도, 효과가 없다는 검정 결과도 아니다. NA는 특정 지표를 계산할 분모가 없거나 정의할 수 없다는 뜻이다. 빈 보존 코호트의 실패율은 NA이며, 모델 실행 자체가 없는 자연 발생률은 NR이다. 데이터가 없는데 이 둘을 모두 0으로 쓰면 독자가 '관측된 실패 없음'으로 오해한다.

첫 주 원고의 목표는 강한 성능 주장을 제출하는 것이 아니다. 연구가 실제로 실행될 때 어떤 결과가 어떤 주장을 지지하거나 반박할지 미리 고정하는 것이다. 따라서 원고에는 구현된 채점·패킷·검증 기능과, 아직 필요한 자연 후보 생성·모델 실행·정보 격리·토큰 대조를 구분한다. 이 경계를 유지해야 이후의 작은 파일 변경이나 몇 번의 성공이 연구 전체의 성공으로 확대되지 않는다.

2. 이미 알려진 간섭과 이번 연구의 좁은 기여

스킬이 항상 도움이 되는지, 여러 지침이 같은 컨텍스트에서 서로의 행동을 바꾸는지는 이미 연구되고 있다. SkillsBench는 다양한 과제에서 스킬 사용 조건을 비교한다 [1]. Signal or Noise?는 지침 주입의 효과와 길이 대조를 다루고 [2], Instruction Bleed는 함께 놓인 모듈의 내용·양·형식 변경에 따른 행동 변화를 다룬다 [3]. 그러므로 "스킬이 성능을 해칠 수 있다"는 현상 자체를 처음 발견한다는 주장은 하지 않는다.

업데이트를 검증한 뒤 합치거나 기존 행동을 보존하는 발상도 선행연구와 직접 겹친다. SkillCommit은 원래 사례의 재생, 다른 사례로의 행동 전달 확인, 합친 추상화의 원 사례 보존 검증을 명시한다 [4]. SkillSmith는 스킬의 상호작용, 도구의 타입 계약, 호출자와 인터페이스의 동시 변경, 버전 범위를 가진 실패 기억을 다룬다 [5]. SkillEvoReg 초록은 스킬 진화의 과적합과 후보별 회귀 검증을 제시한다 [6]. 특히 SkillSmith의 인터페이스 정리를 확인했으므로, 뒤의 단위 예시를 새로운 일반 이론으로 제시하지 않는다.

이번 설계가 추가로 정확히 고정하려는 것은 후보를 만든 뒤 개별 개선을 통과한 동일 조상 가지들을 고정하고, 새 실행에서 조합 효과를 측정하는 실험 단위다. 후보 생성, 개별 자격화, 보존 코호트 선정, 통합 정책 개발, 최종 평가를 서로 구분한다. 그 덕분에 유리한 후보만 골랐는지, 통합 정책이 정답을 미리 보았는지, 반복 횟수를 독립 표본처럼 부풀렸는지 검사할 수 있다. 이것은 기여 후보의 범위이며, 모든 선행 시스템에 없는 요소라고 인증한 결과는 아니다.

기여의 우선순위도 나눈다. 첫째는 관측 가능한 비합성 행동의 측정이다. 둘째는 한정된 검증 예산으로 어떤 조합·보수·보류를 선택할지 비교하는 문제다. 셋째는 통합 결정에 따라 다음 후보까지 달라지는 계속 진화와 메타 정책 문제다. 셋째 연구를 첫째 실험의 성능표에 얹으면 후보군 자체가 달라진 원인을 통합 방법의 효과로 오인하기 쉽다. 첫 주에는 첫째의 식별 가능한 프로토콜을 완성하고, 나머지는 별도 조건부 계획으로 남긴다.

공식 원문 검토 범위는 SOURCES.json에 적는다. 일부 자료는 초록만, 일부는 HTML의 설정·방법·정리절을 읽었다. 저장소 전체와 실행 로그를 모두 비교한 것은 아니다. 논문의 주장대로 동작하는지 재현하지 않았으며, 우선권·정확성·심사 통과를 판정하지 않는다. 추가 검토에서 같은 프로토콜을 직접 수행한 선행연구가 발견되면 연구의 차별점도 다시 좁혀야 한다.

3. 통합 사건, 공통 조상, 독립 작성의 의미

하나의 통합 사건 e는 파일 두 개보다 훨씬 많은 정보를 포함한다. 같은 조상 상태 S0, A와 B의 후보 해시, 각 작성 이력, 모델과 실행기 버전, 과제 계약, 도구 권한, 자료 마운트, 활성화 순서, 자원 한도, Q/C/S 패널을 함께 기록한다. 동일한 SKILL.md라도 모델이나 자료 접근 권한이 바뀌면 같은 처리 조건이 아니다. 과거 성공 기록을 현재 상태의 새 성공처럼 재사용할 수도 없다.

S0는 고정한 출발점이다. A는 S0에 한 업데이트를 적용한 상태이고, B도 같은 S0에 다른 업데이트를 적용한 상태다. B를 만들 때 이미 A를 적용했다면 두 가지가 같은 조상에서 독립적으로 갈라진 비교가 아니다. 그 경우는 순차 재기반과 재검증 연구로 이름을 바꿔야 한다. A+B도 단순히 A 파일 다음에 B 파일을 붙인다는 뜻인지, 서로 다른 단계의 모듈을 교체한다는 뜻인지, 라우터가 둘을 선택할 수 있게 만든다는 뜻인지 명시한다.

조합 생성 함수도 사전에 고정한다. 파일 전체를 대체하는 업데이트와 새 모듈을 추가하는 업데이트는 다르다. B를 마지막에 복사해 A의 내용을 지워 버린 상태를 두 후보가 그대로 함께 사용되는 상태로 부르지 않는다. 같은 파일의 겹치는 diff, 동일 name/trigger, 중복 자료의 충돌은 별도 integration 기록을 남기고 정해진 처리 규칙으로 해결하거나 보류한다. 두 후보의 정확한 내용이 설치·이용 가능 상태에 남는지 artifact hash로 확인한다. 실제 router가 한쪽을 부르지 않은 경우는 availability 결과에 포함하지만, 그때 두 내용이 실행 중 의미 충돌을 일으켰다고 추론하지 않는다.

독립 작성은 정보 접근의 독립성을 뜻한다. 두 작성 경로는 상대의 초안·수정 diff·검증 결과·실패 교훈을 볼 수 없어야 한다. 동일 모델을 다른 seed로 두 번 호출했다는 설명만으로는 이를 보장하지 못한다. 공용 대화 이력, 캐시, 공유 작업 파일, 도구 로그를 통해 상대 후보의 내용을 읽었다면 격리가 깨진다. 후보 입력 목록, 접근 기록, 종료 시점과 예산 기록이 필요한 이유다.

이와 별개로 효과의 통계적 독립성은 보장되지 않는다. 같은 모델이 같은 오류를 내거나 같은 과제 분포를 보았기 때문에 A와 B의 결과가 상관될 수 있다. 오히려 연구는 그 상관을 유지한 채 동일 과제에서 가지별 결과를 비교해야 한다. "각각 작성했으니 각 실행을 독립 표본으로 계산한다"는 결론은 나오지 않는다.

사건의 전체 분모에는 제안 실패와 개별 자격 실패를 포함한다. 교육용 가상 예로 100쌍을 생성해 20쌍만 Q를 통과했다고 하자. 통과한 20쌍의 조합 효과는 'Q 통과 조건부' 결과이지, 모든 100쌍이나 임의 스킬 라이브러리의 효과가 아니다. 나머지 80쌍을 버리고 "발견한 사례 중 대부분이 충돌했다"라고 쓰면 표본 선택 과정을 숨긴다. A만 통과, B만 통과, 둘 다 실패, 작성 중단을 각각 보존해야 한다.

현재 v3의 후보는 한 설계자가 작성한 파일이다. 실제 SKILL.md 바이트와 해시는 있지만, 서로 차단된 작성 경로의 경험적 개선 이력이 없다. 따라서 'crafted candidate'라고 기록한다. 실제 후보를 만들고 Q에서 각각 개선을 확인한 이후에만 연구의 eligible event라는 표지를 붙인다. 파일명에 'improved'가 들어 있다는 이유로 자격을 부여하지 않는다.

4. 과제 계약과 중간 계약을 구분하기

최종 과제 계약은 사용자가 요구하는 결과다. 청구 과제라면 원화 금액, 단위 변환, 항목 ID, 총합과 반올림 규칙이다. SQL 과제라면 반환 행의 의미, NULL과 중복 처리, 순서 여부가 포함된다. 증거 과제라면 주장 판정, 원문 연결, 허용된 근거 표기 규칙이 된다. 이 계약은 S0/A/B/AB의 모든 arm에서 동일해야 한다. 합친 상태만 더 엄격하게 채점하면 조합 손실이 아닌 채점 변경을 측정하게 된다.

중간 계약은 한 단계가 다른 단계에 전달하는 표현의 의미다. 숫자 10만 전달하면 '10원', '10천원', '10달러' 중 무엇인지 알 수 없다. amount=10, unit=thousand_won이라는 레코드는 단위 의미를 보존한다. 수신 단계가 이미 원화로 정규화된 값만 받는다고 가정하면 두 표현은 호환되지 않는다. 문법상 JSON이고 숫자 필드가 있다는 사실만으로 의미가 맞는 것은 아니다.

출처도 비슷하다. 배열의 첫 번째 위치를 source ID로 간주하는 지침과 원본 문서의 불변 ID를 유지하라는 지침은 각각 특정 조상 처리기에서는 성공할 수 있다. 두 지침을 조합해 정렬 후의 배열 위치를 원본 ID로 쓰면 출처가 바뀐다. 이런 문제를 알려면 최종 정답 외에 중간 ID 매핑과 정렬 전후의 자료 해시를 확인해야 한다. 다만 이러한 이야기는 가능한 실패 모형이며 실제 LLM에서 발생했다고 보고하지 않는다.

계약을 더 정밀하게 표현할 때 전제조건과 사후조건을 쓴다. 전제조건은 "입력 레코드는 원화 정수이며 원본 ID를 보존한다"이다. 사후조건은 "출력 금액과 총합이 맞고 필수 ID를 포함한다"이다. 두 단계가 이어지려면 앞 단계의 사후조건이 뒤 단계의 전제조건을 충족해야 한다. '둘 다 자기 시험을 통과했다'는 조건은 이 연결 관계를 검사하지 않는다. 각자의 시험 상대가 원래 조상 모듈이었기 때문이다.

중요한 구분은 행동 실패와 계약 충돌 원인이다. AB가 실패하면 행동 패턴은 확인할 수 있다. 그러나 이유가 단위 충돌인지, 긴 지침 때문에 필요한 부분을 놓쳤는지, 활성화가 안 되었는지는 아직 모른다. 계약 설명은 확인 가능한 중간 레코드, 위반 항목, 관련된 통제 개입을 가져야 한다. 단순한 실패 해설이나 모델의 확신은 원인 인증이 아니다.

계약이 완전하지 않을 때는 채점기의 성공도 제한적이다. 파일 재구성의 AST 검사는 반환 코드의 구조를 확인할 수 있지만 실제 import 실행 성공을 보장하지 않는다. 증거 문장의 정확한 인용은 문장이 주장을 함의한다는 뜻과 다르다. 연구는 "공개 계약의 이 항목을 보존했다"는 범위로 주장해야 한다. 모든 실무 안전성과 사실 정확성을 자동으로 포괄하지 않는다.

5. 직접 계산하는 세 입력 반례: (1, 2, 2, 0)

이 절은 교육용 저자 결정론적 모형이다. 실제 모델 응답, 자연 발생 빈도, 정보 격리된 후보 생성의 관측이 아니다. 범위를 세 개의 유효 입력으로 제한해, 개별 평균 개선과 조합 성공이 논리적으로 다른 조건임을 계산한다. 금액 파싱은 이 입력 집합에서만 정의하며, 임의의 잘못된 문자열을 처리하는 완성된 금융 파서는 아니다.

공개 입력 x는 금액 문자열 text, 단위 unit, 원본 record_id, 최종 ID가 필수인지 나타내는 require_id를 가진다. 각 arm은 같은 x를 받는다. 최종 판정 Φ는 금액을 정확한 원화로 반환하고, require_id가 참이면 원본 ID도 그대로 반환할 것을 요구한다. 정답 금액은 평가자에게 있다. 어느 변환 함수에도 숨은 정답 값을 전달하지 않는다.

원래 읽기 R0는 쉼표가 없는 정수 문자열을 읽어 원화로 정규화한 Won 레코드를 만든다. 쉼표가 있으면 오류다. A의 읽기 RA는 이 모형의 쉼표를 제거해 파싱하되, 원래 숫자와 단위를 유지한 Native 레코드를 만든다. 두 읽기는 모두 원본 ID를 중간 레코드에 넣는다. A는 파싱 범위를 넓혔지만 내부 출력 표현도 함께 바꾼 셈이다.

원래 보고기 P0는 Won과 Native를 모두 받고 필요한 단위 변환을 수행한다. 정확한 금액을 반환하지만 최종 record_id는 생략한다. B의 보고기 PB는 Won만 받으며 금액과 record_id를 모두 반환한다. PB가 ID를 알기 위해 평가 정답이나 특별한 공개 입력을 추가로 받는 것은 아니다. R0가 이미 전달한 중간 ID를 복사한다. RA도 ID를 전달하지만 그 레코드의 Native 타입이 PB의 입력 계약에 맞지 않는다.

입력	공개 문자열·단위	ID 필수	평가 금액	주된 경계
t0	10, 천원	아니오	10,000원	원래 풀던 단위 과제
tA	1,000, 원	아니오	1,000원	R0의 쉼표 파싱 오류
tB	10, 원	예, rb	10원과 ID rb	P0의 최종 ID 생략

입력	S0=P0∘R0	A=P0∘RA	B=PB∘R0	AB=PB∘RA	단위 어댑터 뒤 PB
t0	1	1	1	0	1
tA	0	1	0	0	1
tB	0	0	1	0	1
세 입력의 성공 개수	1	2	2	0	3

t0에서 R0는 Won(10000, r0)을 만들고 RA는 Native(10, 천원, r0)을 만든다. P0는 어느 레코드든 10000원으로 바르게 보고한다. PB는 R0의 Won을 받아 성공한다. 그러나 AB에서는 Native가 PB에 도착하므로 거절된다. 따라서 t0의 성공 패턴은 정확히 (1,1,1,0)이다. 명시적 거절은 틀린 10원을 조용히 반환하는 것보다 나을 수 있지만, 이 과제의 완료 지표에서는 실패다.

tA에서는 R0가 쉼표 때문에 중간 레코드를 만들지 못한다. RA와 P0는 성공하므로 A만 해당 결함을 고친다. tB에서는 두 읽기가 금액과 ID를 정확히 만들지만 P0가 마지막 ID를 버린다. PB는 R0의 ID를 유지하므로 B만 해당 결함을 고친다. '개별 개선'이 모든 과제에서 더 높은 성공을 뜻할 필요가 없다는 점도 드러난다.

세 입력에 균등한 확률을 주면 S0 평균은 1/3, A와 B는 각각 2/3, AB는 0이다. 각 단일 업데이트의 평균 개선은 +1/3이지만 조합은 보존 과제 t0까지 잃는다. 이 계산은 개별 개선이 조합 개선을 함의하지 않는 반례다. 실제 연구의 Q/S 분리 조건이나 작성 독립성을 충족한 사건 수로 세지 않는다. 여기의 입력 표는 수학 모형을 정의한 것이며, 모델 평가용 숨은 시험 문제로 배포하지 않는다.

어댑터 T는 Native의 실제 단위를 읽어 정확한 Won으로 바꾸고 ID를 유지한다. PB∘T∘RA는 세 입력을 모두 해결한다. 반대로 타입 표지만 Won으로 바꾸면서 숫자 10을 그대로 두는 '태그 지우기'는 t0를 10원으로 오해하게 한다. 안전한 타입 연결에는 숫자뿐 아니라 단위 의미의 보존이 필요하다. 이 결과를 근거로 실제 LLM에서도 어댑터가 반드시 100% 성공한다고 말할 수는 없다.

6. 보존 조건의 가정과 짧은 증명

앞 절의 반례는 "연결부가 맞지 않아도 개별 시험은 통과할 수 있다"는 계산이다. 그 반대 방향으로, 어떤 가정이면 합성 성공을 보장할 수 있는지도 분명히 적는다. 이 절은 저자가 명시한 결정론적 충분조건이며, LLM의 모든 행동이나 선행연구보다 새로운 정리를 주장하지 않는다. 조건이 실제 시스템에서 성립하는지 확인하는 작업이 연구의 실증 부분이다.

입력 영역 D, 앞 처리기 RA, 뒤 처리기 PB, 최종 과제 판정 Φ를 고정한다. Pre(PB,x)는 PB가 받을 수 있는 중간 레코드 가운데 입력 x의 의미를 정확히 보존한 집합이다. 앞 예시에서는 단순히 kind=Won인 모든 숫자가 아니다. 실제 입력의 단위에 맞게 정규화한 금액과 필요한 원본 ID를 가진 Won이어야 한다. 잘못된 10원을 Won이라고 적는 것은 문법 타입을 통과해도 이 의미 전제를 충족하지 않는다.

$$\forall x \in D : R_{A(x)} \in \mathrm{Pre}(P_B, x)$$

첫 번째 가정은 RA가 모든 x에서 오류 없이 이 집합의 값을 만든다는 것이다. 두 번째 가정은 PB가 이 집합의 어떤 값을 받더라도 Φ를 충족하는 최종 출력을 만든다는 것이다. 즉, 앞 단계가 표현 의미를 바르게 보존하고 뒤 단계도 그 의미를 완성 결과로 보존해야 한다.

$$\forall x \in D, \forall m \in \mathrm{Pre}(P_B, x) : \Phi\left(x, P_{B(m)}\right)$$

증명은 짧다. 임의의 x를 고른다. 첫 번째 가정으로 m=RA(x)는 PB의 의미 전제 집합에 들어간다. 두 번째 가정에 바로 이 m을 넣으면 Φ(x,PB(RA(x)))가 성립한다. x를 특별히 고르지 않았으므로 D 전체에서 성립한다. 단위 어댑터를 쓰는 경우 앞 처리기는 T∘RA로 바꿔 같은 논리를 적용한다.

$$\forall x \in D : \Phi\left(x, P_{B\left(R_{A(x)}\right)}\right)$$

이 조건은 충분조건이다. 입력 타입이 다르다고 항상 최종 실패하는 것은 아니다. PB가 원 입력을 보고 스스로 복구하거나, 다른 경로가 정답을 만들 수도 있다. 반대로 타입이 같아도 단위·식별자·출처·상태 의미를 틀리게 보존하면 실패할 수 있다. 따라서 'JSON 스키마 일치율'과 '계약 의미 보존율'을 구별해야 한다.

LLM에서는 처리기가 확률적이고, 지침을 읽었는지 불완전하게 관측되며, 이전 메시지·도구 출력·컨텍스트 제한이 영향을 준다. 이 모형에 그런 상황을 그대로 대입해 전제의 참을 선언할 수 없다. 실제 주장에는 중간 산출물의 의미 검사와 새 실행 평가가 필요하다. 완전한 계약을 이미 만족한다는 가정을 넣고 성공을 결론내는 정리는, 그 가정을 측정하지 못하면 성능 보장이 아니다.

또한 원래 계약을 바꾸는 보수는 별도 처리다. 예를 들어 ID 요구를 없애거나 허용 오차를 크게 늘리면 기존 과제를 바르게 보존한 것이 아니다. 정당한 계약 변경이 필요하다면 새 과제 버전과 새 결과를 만들고 이전의 완료율과 직접 섞지 않는다. 이 연구의 보수 개입은 최종 계약을 그대로 두고 내부 표현만 맞추는 조건을 먼저 시험한다.

7. θ, τ, strict witness: 세 수치가 말하는 것이 다르다

주 추정량 θ는 Q를 통과한 동일 후보 쌍에서 조합 상태의 성공확률이 단일 상태들의 평균보다 얼마나 높은지를 나타낸다. p0는 조상, pA는 A만 적용, pB는 B만 적용, pJ는 사전에 정한 joint 순서의 확률이다. e는 후보 생성·수정 이력, t는 과제다. 각 p는 새 실행 S의 확률이며, Q에서 얻은 성공률을 그대로 가져오지 않는다.

$$d_{e,t} = p_J - \frac{p_A + p_B}{2}$$

$$\theta = E\left[d_{e,t} \mid Q_e = 1\right]$$

단일군 평균을 기준으로 삼는 것은 'A와 B 가운데 동전으로 하나를 배포하는 정책'과 비교하는 의미를 갖는다. 어느 쪽이 실제로 더 좋은지 모르는 상황에서 사후 선택 편향을 줄이기 위한 고정 기준이다. S 결과를 본 다음 가장 높은 단일군을 고르면 그 최대값에는 표본 잡음까지 포함된다. 실제 배포의 강한 단일 선택 정책을 비교하려면 D나 Q의 정해진 규칙으로 미리 선택하고 S에서 다시 평가한다. 그 비교는 별도 이차 추정량으로 기록한다.

교육용 가상 예로 p0=.20, pA=.70, pB=.70, pJ=.60이면 θ의 해당 사건-과제 값은 −.10이다. 조합이 단일군들보다 10%p 낮다는 뜻이다. 그러나 조상보다 여전히 40%p 높다. 따라서 '조합의 추가 손실'과 '조상 대비 업데이트 전체의 해'는 다르다. p0를 함께 보고, 기존 능력과 새 능력 코호트를 나누는 이유다.

τ는 2×2 상호작용 대비다. 조상 대비 A의 변화와 B의 변화가 단순히 더해질 것이라는 기준에서 joint가 얼마나 벗어나는지 계산한다.

$$\tau = p_J - p_A - p_B + p_0$$

교육용 가상 값 (.80,.95,.95,1.00)을 넣으면 τ=1−.95−.95+.80=−.10이다. joint가 모든 단일군보다 성공확률이 높아도 τ는 음수다. 성공확률은 1을 넘을 수 없기 때문에 개별 +.15 개선 두 개를 단순히 더한 1.10은 실현할 수 없다. 그러므로 음수 τ만으로 "합쳐서 무너졌다"는 표지를 붙이면 천장 효과를 실패로 잘못 부른다. 이 값에서 d는 +.05이다.

strict witness는 더 좁은 행동 패턴이다. 결정론적 과제에서는 (Y0,YA,YB,YJ)=(1,1,1,0)을 요구한다. 이 모형의 t0가 해당한다. 확률적 LLM에는 성공 한 번과 실패 한 번으로 이 표지를 붙이지 않는다. 높은 성공 문턱 h와 낮은 성공 문턱 l, 각 확률의 불확실성 판정 규칙을 미리 정한다. 그 후 조상·A·B가 높은 성공 구간이고 joint가 낮은 구간인지 확인한다. 현재 h/l과 반복 수는 미선정이다.

충분한 반복이 없어 판정할 수 없는 경우는 unresolved로 남긴다. 이를 빼고 남은 확실한 사례만 분모로 계산한 비율은 '판정 가능한 사례 중 비율'이다. 전체 assigned 사건-과제 분모의 하한·상한과 unresolved 수를 함께 보고해야 자연 빈도와 구분할 수 있다. AB와 BA 가운데 실패한 순서를 나중에 고르는 것도 금지한다. canonical 순서는 사전 고정하고 BA는 별도로 보고한다.

조상은 못 풀지만 A/B가 새로 푼 문제를 joint가 잃는 현상은 신규 능력 손실이다. 중요하지만 strict1110와 같지 않다. 이 패턴을 strict witness 조건에 억지로 넣지 않고 G 보존과 이차 진단으로 보고한다. 세 지표를 함께 보면 평균 개선, 기존 완료 과제의 붕괴, 신규 획득의 상실을 분리할 수 있다.

8. Q·C·S·D를 분리해야 선택 편향을 볼 수 있다

D는 개발 피드백이다. 후보 작성자가 실패를 보고 문장을 고치거나, 통합 정책이 어떤 재생을 할지 조정할 때 사용한다. 이런 적응은 허용된 연구 행동이지만 시도 횟수와 접근 정보를 남겨야 한다. D를 여러 번 보았다고 그 결과를 최종 독립 평가로 부를 수 없다. 후보 작성과 통합 정책 개발은 모두 개발 영역에서 이루어지되 서로의 정보 접근 경계를 기록한다.

Q는 개별 개선 자격화다. 후보를 고정한 뒤 작성에서 보지 못한 패널에서 S0 대비 A와 B가 각각 개선되었는지 본다. 단순 동률이나 빈 Q는 개선이 아니다. 문턱, 기존 능력 보존 조건, 자원 제한, 불확실성 규칙을 함께 충족해야 한다. Q 결과를 받아 다시 편집하면 그 Q는 사실상 개발 자료로 사용된 것이다. 새로 남겨 둔 Q가 필요하거나, 반복 자격화와 선택의 다중성을 미리 통제해야 한다.

C는 보존 코호트 선정용 독립 실행이다. 여기서 H는 조상이 잘 푸는 과제, G는 조상은 못 풀지만 단일 가지가 새로 푸는 과제로 선정한다. 코호트 문턱은 확률적 실행에 맞춰 사전 고정한다. C의 성공·실패를 S의 성공률 계산에 다시 넣지 않는다. 예를 들어 C에서 성공한 과제만 H로 뽑은 뒤 그 성공을 그대로 baseline=100%로 쓰면, 잡음 때문에 운 좋게 뽑힌 과제도 완전한 기존 능력으로 취급한다.

S는 최종 새 실행이다. 같은 H 과제에서 조상도 다시 실행하고, 통합 정책이 선택한 상태도 다시 실행한다. 그러면 C에서 성공해 뽑힌 조상의 S 성공률이 100%보다 낮을 수 있다. 이는 자동으로 능력 상실이나 환경 오류를 뜻하지 않는다. 선택 후 평균으로 돌아가는 효과와 원래 모델의 확률적 실패가 포함된다. 그래서 절대 실패율과 baseline 대비 초과 손실을 따로 계산한다.

C와 S가 같은 입력의 다른 실행인 설계는 그 입력에 대한 보존을 측정한다. 새로운 단위값·새 문서·새 저장소로 바꿨다면 이전 과제와 동일한 hash의 replay가 아니다. 그때는 같은 family의 일반화 또는

능력 전달로 추정량을 바꿔야 한다. fresh seed는 관측을 새로 만들 수 있지만 입력의 낯섦을 만들거나 과거 답의 노출을 지우지 않는다.

현재 SciFact의 train/dev/test 306/100/125를 활용하는 구체적 제안은 다음과 같다. train 내부의 원본 connected component들을 다시 나눠 후보 작성과 정책 D에 배정한다. dev 100개는 작성자나 정책 조정에 보여 주지 않는 Q로 예약한다. test 125개의 과제 hash는 C의 코호트 실행과 S의 독립 새 실행에 사용한다. C가 어떤 test 과제를 H/G로 뽑았는지, 어떤 답을 냈는지는 정책에 넘기지 않는다. 이것은 아직 unlocked 역할 배정 제안이며, 실제 Q/C/S model 기록이 만들어진 것은 아니다.

역할	볼 수 있는 정보	넘기면 안 되는 정보
후보 작성	조상과 지정 D 입력·피드백	상대 후보·Q/C/S 정답·결과
통합 정책	고정 후보·지정 D 근거·공개 계약	C 코호트 판정·S 결과·숨은 예상 출력
Q 평가자	frozen 후보와 Q 정답	Q 결과를 통한 무기록 재편집
C/S 평가자	해당 실행과 evaluator-only 라벨	정책으로의 정답·최종 성능 피드백

Q를 dev 100개로 쓰면 같은 dev를 정책 선택에도 사용할 수 없다. 문서상의 파일이 세 폴더로 나뉘었다는 것만으로 이러한 권한 분리가 이루어지지 않는다. 현재 로컬 자료는 공개 데이터이고 프로세스 수준에서 모두 읽을 수 있다. 실제 시험 전에 runner의 마운트·접근 기록·평가 서비스와 후보 세션 경계를 검증해야 한다.

한편 주 θ는 Q-eligible 쌍의 사전 배정 과제 전체에서 계산하고, C가 선택한 H/G만으로 바꾸지 않는다. H/G 분석은 설명과 보존 지표의 이차 분석이다. AB가 실패한 과제만 C에서 골라 "충돌 빈도"를 측정한다면 실패 사례 모음의 비율이며 자연 발생률이 아니다. Q의 선택 자체도 모집단을 조건화하므로 all-attempt와 eligible의 수를 끝까지 나란히 보고한다.

9. 짝지음, 과제 가족, 후보 이력과 실제 표본 수

paired 비교는 같은 입력·계약·환경에서 S0/A/B/AB의 결과를 비교한다는 뜻이다. 과제가 다른 두 arm의 평균만 비교하면 난이도 차이가 섞일 수 있다. 동일 과제의 반복 배정을 맞추면 과제 난이도는 비교에서 유지되지만 모델의 모든 행동을 동일하게 만들 수는 없다. 같은 seed라도 지침이 바뀌면 다음 토큰과 도구 경로가 달라지므로 '같은 궤적에서 스킬 효과만 바뀌었다'는 해석은 하지 않는다.

최소 실행 행은 task×event×arm×order×repetition이다. 그러나 연구 표본 단위는 이 행과 같지 않다. 같은 원문 문서의 숫자만 바꾼 입력, 같은 저장소의 유사 이슈, 같은 SQL DB의 질문들은 의존할 수 있다. 이들은 source family나 connected component에 묶는다. 같은 skill update history로 여러 과제를 실행한 결과도 서로 연결된다. 가족과 이력이라는 두 축의 의존성을 함께 생각해야 한다.

교육용 가상 예로 10개 원본 workflow, 5개 후보 생성 이력, 4개 arm, 20회 반복이면 4,000행이 생긴다. 이 숫자는 과학적으로 독립적인 4,000개의 현상 사례가 아니다. 과제의 종류는 10 family이고 후보 생성 경로는 5 history다. 실행 반복은 각 셀의 성공확률 불확실성을 줄이는 자료다. family의 낯섦이나 새로운 업데이트의 다양성을 늘리는 자료는 아니다.

특히 한 후보 쌍을 모든 family에서 평가하고 다른 쌍도 같은 family를 사용하면 의존성이 교차된다. family만 bootstrap하면 후보 생성 차이를 놓칠 수 있고, history만 bootstrap하면 과제 family 차이를 놓칠 수 있다. 하나의 평면 그룹 이름을 만들어 기존 분석 함수를 호출하는 것만으로 해결되었다고 선언하지 않는다. 어떤 분포로 일반화하려는지와 각 축의 유효 cluster 수를 먼저 정해야 한다.

계획의 선택지는 두 가지다. 독립 family·history 블록을 의도적으로 배정해 보수적으로 block-level paired 차이를 분석하거나, 충분한 양쪽 cluster를 확보한 후 교차 의존성을 다루는 통계 모형·resampling을 검토한다. 연결된 모든 행을 무조건 한 cluster로 묶는 방식은 매우 보수적일 수 있지만 정보가 거의 없어질 수도 있다. 반대로 연결을 무시해 좁은 CI를 얻는 것도 정당하지 않다. 현재 최종 분석 방식과 표본 수는 고정되지 않았다.

가중치는 추정하고 싶은 모집단의 정의다. 100문제를 가진 family와 2문제를 가진 family에 각각 같은 무게를 주면 equal-family 효과를 측정한다. 문제별로 같은 무게를 주면 큰 family가 더 큰 비중을 가진다. 두 방식은 서로 다른 질문이며 어느 쪽이 정답인지는 실제 배포 분포에 달려 있다. 이력이 많이 생성된 family를 자동으로 과대 대표하지 않도록 family와 history의 가중치를 고정한다.

희귀 strict witness를 발견하려면 반복 수뿐 아니라 eligible 사건 수가 필요하다. Q 통과율이 낮으면 생성 1,000회도 분석 가능한 쌍이 적을 수 있다. 높은 성공 문턱을 확률적으로 인증하려면 한두 회 성공으로 부족하다. pilot에서는 통과율, family별 변동, 반복 간 상관, 판단 보류율을 먼저 얻고, 필요

한 정밀도나 최소 의미 효과를 만족할 실제 예산을 시뮬레이션한다. 현재 fixture의 네 family로 모집단 CI나 검정력을 추정했다고 쓰지 않는다.

10. 주실험 SkillsBench v1.1을 어떻게 연결할 것인가

주실험 환경은 SkillsBench v1.1이다. 공식 현재 inventory는 native task 87개, 8개 범주이며 14개 extra task는 주 roster와 분리한다. 이 숫자는 문헌·release metadata의 목록 수다. Q/S 분할과 원본 family 감사가 끝나지 않았으므로 최종 S의 과제 수나 독립 family 수가 87이라는 뜻은 아니다. 원고의 최신 SkillsBench 논문은 v4를 인용하고 이전 버전의 목록 수를 혼합하지 않는다 [1].

두 핀의 역할을 분리한다. release/tooling commit은 b63b7b2850226b6aa4fb5929a8c1ac7bc4d9a6af이고 registry가 가리키는 task-tree commit은 55bfe693f2a19f6b2f29aca3f54fe98b9d994668이다. 하나는 평가 도구·등록 설정의 버전이고 다른 하나는 과제 내용의 버전이다. 현재 파일을 내려받아 로컬로 확인한 task archive나 image digest는 없으므로 URL 핀을 실제 확보 해시 인증으로 쓰지 않는다.

native task.md, 공개 입력 자료, 환경 정의, oracle, verifier를 따로 inventory에 넣어야 한다. agent에는 공개 과제·허용 자료·후보 지침만 보이고 oracle과 숨은 기대 결과는 평가자에게 남긴다. native verifier를 실제로 실행하기 전에는 환경 준비가 됐다고 표시하지 않는다. clean reset도 계획이 아니라 검사 대상이다. arm 사이 파일, 성공 산출물, 캐시와 이전 도구 로그가 남는지 확인해야 한다.

스킬 후보는 기존 bundled skill을 아무 검토 없이 복사하기보다 연구용 새 지침으로 작성한다. 저장소 코드의 Apache 허가는 모든 외부 문서·데이터·번들 자산의 권리 확인을 대신하지 않는다. 검토한 한 PDF 스킬은 별도 proprietary 표기를 가지고 있었고 완전한 자산 권리는 아직 확인하지 못했다. 따라서 현재 번들 재배포는 하지 않으며, 후보 작성과 입력 자료 사용의 조건을 각 자산별로 기록한다.

실험 설계 예는 '파일에서 중간 구조를 추출하는 가지'와 '그 구조를 집계하고 보고하는 가지'다. A 작성자에게는 누락 없이 단위·키·출처를 보존하는 개선 목표를, B 작성자에게는 최종 보고의 집계·누락·형식 오류를 개선하는 목표를 준다. 어느 쪽에도 "서로 충돌하도록 표현을 바꾸라"는 지시를 주지 않는다. 각자의 수정이 실제 Q를 통과한 후 원래 public final contract로 joint를 평가한다. 이 목표는 새로운 authoring 설계이며 원본 task 지침의 인용이나 이미 관측한 SkillsBench 업데이트가 아니다.

분할은 원본 workflow와 자료 연결을 먼저 확인한 뒤 만든다. 서로 다른 task 이름이 같은 입력 패키지나 후속 단계라면 같은 source family로 유지할 수 있다. Q가 필요하므로 roster의 일부를 자격화에 배정하면 최종 held-out 목록은 줄어든다. 이 문제를 해결하려고 S 정답을 후보 개발에 쓰면 안 된다. 새로 만든 variant를 Q에 쓰는 대안은 별도 adapted track으로 표시하고 native benchmark result와 구분한다.

SkillsBench의 native with/no-skill 비교가 우리 연구에 필요한 개별 개선 이력을 제공하지는 않는다. S0/A/B/AB 상태, 독립 작성, Q 자격, C/S 관측과 동일 자원을 새 adapter로 연결해야 한다. 현재 metadata review는 완료했지만 이 native adapter·환경 실행·권한 경계는 미완료다. 모든 모델 성능 칸은 NR이다.

11. SciFact 531개 준비 자료의 정확한 의미

SciFact-CitedPair-RFv4는 원본 SciFact의 전체 검색·근거 추출 leaderboard를 그대로 실행하는 데이터가 아니다. 원본 claim과 그 claim이 인용한 abstract 하나를 묶어, 주어진 문서 기준의 판정을 평가하는 로컬 변형이다 [7]. 공개 입력에는 claim, 선택된 문서 ID, title, 문장 목록과 component 식별자가 들어간다. evaluator-only 파일에는 label과 허용 rationale 문장 집합이 있다. 같은 claim의 모든 인용 문서를 검색해 공식 점수를 계산하는 실험과 구분한다.

준비 기록에서 원래 corpus는 5,183개 문서, 공개 train claim은 809개, 공식 dev claim은 300개다. 같은 claim의 모든 cited document 연결과 정규화한 claim 문자열 연결을 사용해 component를 만든다. 공식 dev와 연결된 train component 전체를 제외하므로 train claim 278개가 빠진다. 남은 531개의 claim에서 문서 하나를 선택한 뒤, 같은 component가 새 train/dev/test를 넘나들지 않도록 고정 hash로 분할한다.

새 split	입력 수	component 수	supported / contradicted / insufficient
train	306	193	130 / 60 / 116
dev	100	62	39 / 17 / 44
test	125	70	42 / 21 / 62

새 split	입력 수	component 수	supported / contradicted / insufficient
합계	531	325	211 / 98 / 222

이 표는 실제 준비 산출물의 분할 감사값이다. 성능표가 아니며 531개 독립 LLM 업데이트 사건을 의미하지 않는다. component도 연결 규칙으로 만든 독립성 보호 단위이지 모든 의미상 의존성을 완벽히 발견했다는 보증은 아니다. claim 정규화는 NFKC·casefold·공백 규칙으로 문자열 중복을 잡지만 서로 다른 표현의 의미적 중복은 놓칠 수 있다.

문서 선택은 label을 보기 전에 claim ID와 doc ID의 hash로 결정한다. '정답 근거가 있는 문서만 골라' 과제를 쉬워지게 하는 선택을 피하기 위한 규칙이다. 선택한 문서에 원본 evidence annotation이 없으면 로컬 판정은 insufficient가 된다. 이것은 annotation 기준이며, 그 문서에 실제로 어떠한 관련 근거도 없다는 전면적인 과학적 사실 검증은 아니다. 모호한 사례를 별도 감사할 필요가 남는다.

새 test는 공개 원본 train에서 만든 것이다. 원본 공식 test를 손대지 않고 평가했다거나 사전학습 노출이 없다고 표현하지 않는다. 원본 라벨은 인터넷에서 공개되어 있고 개발자도 자료를 읽을 수 있다. 파일 분리는 모델 입력 패킷에 답을 넣지 않게 해 주지만 비밀성이나 오염 배제를 인증하지 않는다. 모든 결과에는 adapted public-data pilot이라는 조건을 붙인다.

현재 채점기는 schema-valid label_correct를 주 지표로 삼는다. label은 supported/contradicted/insufficient 중 하나이고 문장 번호는 범위 안의 중복 없는 정수 목록이어야 한다. 잘못된 schema·범위·유형이면 주 완료값은 실패다. 응답 task ID와 packet 연결은 외부 원장이 확인해야 한다. 라벨값 하나만 가져와 어떤 과제의 응답인지 연결하지 않으면 결과 집계의 신뢰성이 없다.

근거 문장에는 하나의 원본 정답 집합만 강요하지 않는다. 완전한 annotation alternative 중 하나를 포함하면 coverage를 인정하고 더 넓은 superset도 받아들인다. 그 결과 모든 문장을 선택해도 coverage가 될 수 있다. 정확한 근거 추출을 잘했다고 보려면 annotated precision을 별도로 봐야 한다. coverage와 label을 합친 joint_pass는 진단값이고, native SciFact의 검색·rationale F1이나 의미함의 인증으로 바꾸지 않는다.

교육용 가상 문서로 "실험군에서만 효과가 있었고 대조군에서는 확인되지 않았다"는 두 문장을 생각할 수 있다. A는 조건·대상·부정 표현을 보존하는 추출 지침을 개선하고, B는 판정과 번호를 일관된 출력으로 정리하는 지침을 개선할 수 있다. 두 가지가 각각 Q를 통과한 뒤 조합에서 조건이 떨어지면 관측 대상으로 삼는다. 이 문서는 실제 corpus의 인용문이 아니며, 일부러 조건을 지우게 만든 stress pair를 자연 후보 빈도에 넣지 않는다.

원본 claims/evidence, abstracts, code의 사용 조건은 각각 기록한다. 로컬 SOURCE_LICENSE.md와 NOTICE.json은 원본 선언과 변경 사항을 포함한다. 현재 원본 선언은 claims/evidence CC-BY-4.0, abstracts ODC-By-1.0, code Apache-2.0이고, 별도 mirror card의 이전 단일 표기와 차이가 있다. 어떤 artifact의 어떤 license를 사용했는지 남겨야 한다. 이 기록은 제3자 권리를 전부 심사한 인증이 아니다.

train에 별도로 만든 44개 label-swap 항목은 Second-Chance용 구성된 평가 오류 stress 파일이다. 원본 라벨은 수정하지 않았다. 44개 자연 평가기 결함, 44개 후보 거절 이력, 44개 능력 회복 실험이라는 뜻이 아니다. SkillMerge의 정상 평가기는 원본 annotation 기반 라벨을 사용하고, 이 stress 결과를 자연 스킬 조합 효과의 분모에 합치지 않는다.

12. 네 과제 family가 실제로 무엇을 요구하는가

v3에는 invoice, imports, sql, evidence의 네 과제 family와 총 24개 authored public input이 있다. 각 family에는 조상·A·B의 실제 SKILL.md가 있어 모두 12개 패키지다. 이 파일은 입력 패킷을 만들 때 읽히지만 아직 LLM에 의해 실행·개선 검증된 것은 아니다. 여기서는 기존 prototype의 계약과 다음 실험에 연결할 점을 구체적으로 설명한다.

청구/단위 family는 원·천원·만원을 정확한 원화로 바꾸고 원래 행 ID와 순서, 항목 금액, 정수 총합을 보존한다. 실제 고정 합성 fixture의 한 입력은 1만원, 500원, .001천원이다. 직접 산술은 10,000+500+1=10,501원이다. 이는 준비한 fixture의 계산 확인이며 LLM 응답 정확도 실측이 아니다. A 후보는 원래 숫자와 단위를 중간 쌍으로 유지하고, B 후보는 원래 읽기가 정규화했음을 가정해 재변환을 피하는 지침이다. 그 가정이 joint의 표현과 맞는지 확인하는 조작 점검으로 쓰인다.

단위 확장에는 소수, 0, 음수 허용 여부, 쉼표, 통화 종류, 중간 반올림 위치가 포함될 수 있다. 이때 공개 계약에 변환 계수와 허용 규칙을 먼저 적고 evaluator-only에 해당 결과를 둔다. 갑자기 음수 금액을 넣어 기존 계약에 없던 규칙으로 실패시키면 스킬 조합 손실의 증거가 아니다. 원본 입력만 숫자를 바꾼 variant는 같은 family로 연결한다.

파일 재구성/import family는 요청한 모듈을 새 패키지 위치로 옮기고 모든 호출자가 올바른 전체 모듈 경로를 쓰게 하며 함수의 AST와 package marker를 보존한다. A는 중첩 목적지와 public 함수 정의 보존을, B는 mover가 보고한 모듈 이름을 기반으로 import 갱신을 맡는다. 중간 이름이 새 package 경로를 포함하는지 확인하는 경계가 중요하다. 같은 문자열 이름이라도 이전 디렉터리 기준인지 최종 module identity인지 다를 수 있다.

현재 검사에서는 알 수 없는 반환 Python을 실행하지 않는다. AST와 파일 매핑을 검사하므로 runtime import 성공, 동적 경로, 실제 API 동작을 증명하지 못한다. SWE-bench로 확장할 때는 pinned repository와 native tests를 독립 환경에서 실행해야 한다 [8]. v3의 정적 통과율을 SWE-bench 기능 해결률로 대체하지 않는다. 공개 이슈·base commit은 agent 자료이고 gold patch·test patch·예상 로그는 평가자 영역으로 둔다.

SQL family는 거래 ID를 먼저 deduplicate하고 고객 키로 집계하며, 결제가 없는 active 고객도 LEFT JOIN으로 남겨 0을 보고하도록 요구한다. A는 중복 거래의 합산 오염을 줄이고 B는 0-결제 고객의 누락을 줄이는 역할이다. 중복 제거 단계의 별칭·키와 외부 join 단계의 별칭·키가 일치해야 한다. 각각의 질의가 조상 구조에서는 맞아도 joint에서 NULL 필터나 중복 곱셈이 생길 수 있다는 설계다.

실험을 확장할 때 같은 거래 ID에 다른 금액이 있는 경우를 그냥 추가해서는 안 된다. 어떤 값이 진짜인지 고르는 공개 규칙이 있어야 한다. 행 순서가 계약상 무관한지, 정수/실수와 NULL의 비교가 어떻게 이루어지는지도 고정한다. 한 DB에서 같은 결과를 냈다는 사실은 모든 DB에서 SQL이 동치라는 증명이 아니다. 공식 SQL benchmark로 이식할 때 원본 dialect·schema·허용 함수와 채점 기준을 새로 검토해야 한다.

증거 보고 family는 주장 ID를 유지하고 주어진 source로만 판정하며 실제 원문과 일치하는 quote, Unicode 문자 시작/끝 위치, source byte hash를 연결한다. A는 인용의 조건과 local context를 보존하고 B는 출력 근거를 간결하게 만든다. 줄인 인용에서 숫자나 적용 조건이 사라졌는지 별도로 검사한다. 아직 일반적인 자연어 함의 판정기는 아니므로 crafted minimum-context anchor의 성공을 모든 과학 문서의 충분한 근거로 확대하지 않는다.

이 네 family는 실험 장치가 요구하는 경계를 보여 주는 시작점이다. SQL 문법, 통화 계수, 디렉터리 이름을 바꾼 많은 variant를 만들어도 과학적 다양성이 자동으로 늘지는 않는다. 새로운 원본 문서·workflow·저장소·DB와 후보 이력이 필요하다. 실제 main은 SkillsBench, pilot은 SciFact, 외부 재현은 SWE-bench로 각각 따로 평가하고 결과를 한 평균에 섞지 않는다.

13. 길이·순서·라우팅·자원 대조군의 역할

A+B는 보통 A 하나보다 길다. joint가 실패했다고 즉시 내용 충돌을 원인으로 삼으면 컨텍스트 양의 효과를 놓친다. 목표 후보의 지침 블록과 길이가 맞는 무관련 지침, 그리고 내용량을 채운 padding을 따로 비교한다. 이는 Signal or Noise?의 content/length 구분과 연결되지만 [2], 그 논문의 조건을 복제했다고 주장하려면 실제 byte/token 기준과 자산 마운트를 동일하게 구현했는지 확인해야 한다.

우리 최종 계획에서 길이 기준은 실제 사용 모델의 tokenizer다. UTF-8 bytes나 한글 문자 개수는 토큰 수와 같지 않다. v3에는 byte padding이 있지만 tokenizer-matched control은 구현되지 않았다. 읽는 token budget이 같아도 문단 위치·제목·문체·특수기호가 다르면 format 효과가 남는다. 허용 token 오차, 위치, separator, 반복 패턴을 사전에 고정하고 이를 별도 대조 정보로 기록한다.

무관련 지침도 자동으로 중립적이지 않다. 다른 분야의 스킬이 '항상 검증하라'처럼 일반적 도움을 주거나 장황한 수식으로 실행을 방해할 수 있다. 따라서 relevance 판정을 D에서 감사하고 최종 과제의 정답을 보지 않는 범위에서 후보를 정한다. S 결과를 보고 가장 해로운 무관련 지침을 골라 목표 후보와 비교하면 통제군의 선택 편향을 만든다. padding만의 결과와 의미 있는 무관련 블록의 결과를 함께 보면 해석이 더 명확해진다.

순서도 별도 처리 조건이다. AB와 BA가 다른 결과를 내면 '두 파일이 함께 있다'만으로 처리를 설명할 수 없다. canonical order는 평가 전에 정하고 해당 θ를 primary로 삼는다. BA는 계획된 이차 분석이다. 도구 단계의 A→B 실행 순서와 prompt에서 문단을 B→A로 놓는 순서는 서로 다를 수 있으므로 둘을 따로 로그에 남긴다. post-hoc best-order와 worst-order는 탐색 요약이며 primary 결과로 갈아끼우지 않는다.

라우팅은 지침의 효과와 사용 결정의 효과를 함께 만든다. skill availability arm에 배정된 전체 과제에는 스킬을 실제로 호출하지 않은 과제도 들어간다. 안 부른 과제를 빼면 사용 여부라는 처리 후 변수를 기준으로 표본을 다시 선택하게 된다. 호출률은 원인을 설명하는 진단값으로 유용하지만 조건부 효과의 정의를 새로 적어야 한다. 스킬을 강제로 주입한 조건은 분명한 별도 diagnostic arm이다.

v3 패킷은 모든 등록 지침을 prompt에 붙이는 forced-injection 준비다. 자연 router가 사용자 과제에 맞춰 후보를 발견·선택하는 구현은 없다. main에서 availability를 평가하려면 skill 목록 제공 방식, description과 name, 파일 로딩 시점, context 제한과 미호출 기록을 추가해야 한다. 스킬 이름에 '정답', '개선', '충돌' 등 평가자의 판단을 붙여 유용한 정보를 누출하지 않는다. 기능 설명은 공개 과제 수행에 필요한 내용을 담되 arm 평가는 임의 불변 ID로 연결한다.

자원을 맞추는 instruction-only control에서는 모든 arm이 같은 코드·참고자료·실행 도구·데이터·권한을 가진다. 한쪽에만 단위 변환 스크립트가 있고 다른 쪽은 암산해야 한다면 지침의 문장 효과를 분리하기 어렵다. 별도 native-package 비교에서는 스킬에 붙은 코드와 자료까지 포함할 수 있지만, 그때는 package 전체의 배포 효과라고 보고한다. 내용 대조와 package 효과를 동일한 숫자로 해석하지 않는다.

disjoint-scope pair는 두 가지가 같은 중간 표현을 바꾸지 않을 때의 co-residence 효과를 확인한다. 그러나 지나치게 쉬운 과제나 무관한 스킬을 가져오면 위험한 joint와 공정한 비교가 아니다. 각 가지가 Q에서 유용하고 실제 실행에 노출되는 범위를 확인해야 한다. 겹치는 scope 자체가 실패를 보장하는 것은 아니므로 overlap의 정도와 최종 계약 종류를 미리 분류한다.

계약 원인을 시험할 때는 scoped adapter나 조건문만 바꾼다. 예시의 Native→Won 변환은 단위 의미를 바로잡는 개입이다. 이때 추가 도구·token·권한을 제공한 조건이 원인과 함께 바뀌면 개선의 해석이 모호해진다. 관련 없는 길이·자원 변화가 맞춰진 비교를 설계한다. 중간 산출물은 관찰 가능한 JSON·파일·도구 응답으로 기록하고, 모델 내부 추론을 완전히 읽었다고 주장하지 않는다.

14. 평가기와 데이터가 만드는 가짜 조합 실패

채점기는 스킬의 성능을 측정하는 도구다. 채점 자체가 유효한 답을 거절하면 실제보다 단일군·joint 실패율이 높아진다. 특히 두 arm이 서로 다른 정답 표현을 선호하면 채점기의 편향이 상호작용처럼 보일 수 있다. 따라서 agent 실험 전에 정답 하나의 문자열 일치가 아니라 공개 계약이 허용하는 대안을 독립적으로 만들어 시험해야 한다.

v3에서는 실제로 두 false-negative 결함을 수정한 기록이 있다. SQL authorizer가 MIN/IFNULL 같은 안전 함수로 동일 결과를 만드는 질의를 막았고, evidence 채점은 정확한 더 넓은 원문 인용도 한 개 정답 span과 다르다는 이유로 거절했다. v2는 안전 함수 allowlist와 의미상 필요한 최소 context를 보강했다. 이 기록은 채점 소프트웨어의 수정이지 실제 LLM 조합 효과나 공식 benchmark 결함의 발견 결과가 아니다.

현재 SQL 공개 계약은 지원 범위를 명시한다. 허용 함수는 abs, avg, coalesce, count, ifnull, iif, max, min, nullif, round, sum, total이다. 쓰기·ATTACH·PRAGMA·확장 기능은 거절하고 SQL은 10,000 Unicode 문자, 결과는 100행까지다. VM 진행 callback은 100 instruction 간격으로 실행되고 101번째 callback에서 중단한다. 이는 공개된 harness 자원 제약이며 모든 arm에 같게 적용한다. 전체 SQLite 기능을 지원한다고 말하지 않는다.

SQL 제한은 OS 보안 격리가 아니다. 현재의 작은 in-memory DB와 deny-by-default authorizer가 과제 채점의 범위를 줄이는 것이다. 다른 저장소의 위험 코드를 안전하게 실행하는 기반으로 확대하지 않는다. 실제 benchmark의 native 실행은 별도 환경·권한·연산 한도가 필요하다. 알 수 없는 agent Python을 검사 없이 실행해 현재 AST 평가를 기능 검증으로 바꾸지도 않는다.

evidence 채점은 주장 inventory와 판정값을 먼저 검사하고, source ID·source hash·문자 범위·정확한 quote를 다음에 검사한다. 원문의 올바른 sentence 전체가 필요한 숫자와 조건을 모두 포함한다면 좁은 anchor보다 넓어도 허용한다. 반면 무관한 문장, 숫자만 남기고 조건을 지운 문장, 잘못된 source hash, byte offset을 Unicode 문자 위치로 착각한 범위는 거절한다. 길이가 넓다는 이유만으로 충분한 근거가 되는 것은 아니다.

채점기 도전 자료는 개발자가 만든 정답과 같은 방식으로 복제한 답만으로 채우지 않는다. 다른 질의 작성 방식, claim 순서 변경, 동등한 정규화, boundary 값, 잘못된 유형과 원문 연결을 추가한다. 학습될 수 있는 공개 검사는 개발 영역이고 최종 grader audit에는 별도 사례가 필요하다. 테스트 이름에 'valid'가 붙었다는 이유만으로 의미 동치가 입증되었다고도 쓰지 않는다.

데이터 감사에서는 원본 hash만 보는 것으로 부족하다. label 전에 선택했는지, component split이 겹치지 않는지, metadata의 label counts가 실제 재생 결과와 같은지 다시 계산해야 한다. v4는 입력·annotation에서 SciFact 준비를 semantic replay하고 audit와 manifest를 비교한 기록이 있다. 하지만 이 검증은 annotation의 과학적 진실을 인증하지 않는다. annotation 기준 평가와 실제 claim의 사실성은 다르다.

평가기 수정 후에는 원래 실행을 그대로 보존한다. 새 코드·계약·예상 결과의 hash로 새 run을 만들고, 같은 저장 응답을 재채점한 결과와 agent를 다시 실행한 결과를 분리한다. 전자는 측정 오류 정정이고 후자는 행동의 새 표본이다. SkillMerge에서 이 둘을 혼합하면 grader 개선을 스킬의 개선으로 잘못 세게 된다.

15. 기존 능력 실패율, 초과 회귀, 신규 능력 보존

통합 정책이 새 상태 O를 선택했다면 기존에 잘 풀던 H와 새로 배운 G를 따로 본다. H/G는 C에서 독립적으로 고른 집합이며, 확률적 성공 문턱은 잠가야 한다. 여기서 pO는 O의 새 S 성공확률이다. "기존 과제에서 실패했다"는 표현은 O가 실패했다는 말과 그 실패가 업데이트 때문에 늘었다는 말로 나누어야 한다.

$$F_H = 1 - E_{H[p_O]}$$

$$R_{\text{excess}} = E_{H[p_0 - p_O]}$$

교육용 가상 예로 H에 20개 과제가 있고 새 baseline 실행에서 18개, 새 선택 상태에서 17개를 풀었다고 하자. FH=1−17/20=.15이고 R_excess=(18−17)/20=.05다. 15%는 절대 실패율이며 추가 손실은 5%p다. baseline에도 10% 실패가 있으므로 모두 업데이트가 만든 회귀라고 부를 수 없다. 이 숫자는 실측이 아니다. 어떤 과제에서 두 상태가 같이 실패했는지 없이는 paired 불확실성도 계산할 수 없다.

같은 baseline을 그대로 유지하는 보류 정책이라면 이론적 같은 확률에서 R_excess=0이다. 그러나 baseline이 확률적으로 실패하면 FH는 0이 아닐 수 있다. 실제 두 실행의 표본 성공률은 잡음으로 다를 수 있다. 따라서 '보류는 실패율이 0이다'라는 보고도 틀리다. 비교할 상태와 fresh 실행의 표본 성격을 명시해야 한다.

G는 조상은 잘 못 풀지만 단일 가지가 새로 해결한 과제다. O의 G 성공을 신규 능력 보존으로 기록한다. 모든 업데이트를 거절하는 정책은 H의 초과 회귀를 줄일 수 있지만 G의 기회를 잃는다. 예를 들어 설명용 G 10개에서 baseline 1개, A 8개를 푸는데 O가 baseline이라면 G 성공은 10%다. 이를 '안전성 최고' 한 숫자로만 보고하면 능력 상실을 숨긴다.

G의 기준도 필요하다. A만 성공한 집합과 B만 성공한 집합, 어느 단일군이든 성공한 집합은 서로 다르다. 같은 데이터에서 가장 잘 보이는 정의로 바꾸지 않는다. C의 문턱과 branch 선택을 고정하고, S에서 같은 baseline·branch 기준을 다시 평가해 selection noise를 드러낸다. 이 코호트 성능은 해당 C 선정 규칙에 조건부인 결과다.

빈 H나 빈 G는 지표를 계산할 수 없다. H가 없으면 실패율 0이 아니라 NA와 nH=0을 적는다. G가 아직 감사되지 않았다면 nG=0의 빈 집합과도 구분한다. 누락된 C 실행, unresolved 자격, external interruption이 원인인지 원장에 적는다. 완전한 audit가 없는 정책을 '회귀 없음'으로 승격하지 않는다.

no_gain은 예산을 썼어도 사전 개선 기준을 넘는 상태를 얻지 못한 결과다. deferral은 결정을 보류하거나 조상을 유지한 행동이다. 둘은 같지 않다. 정책이 joint를 받아들여도 no_gain일 수 있고, 유망한 가지가 있지만 검증 예산이 부족해 defer할 수도 있다. 성공한 update만 모아 최종 평균을 내면 검증 실패와 보류를 숨긴다. 전체 배정 사건의 O 상태를 기준으로 completion·비용·보존·보류를 함께 평가한다.

16. 한정 예산의 통합 정책: 무엇을 같게 비교할 것인가

두 번째 연구 질문은 후보를 바꾸지 않은 채 어떤 검증을 먼저 하고 어떤 상태를 배포할지 선택하는 문제다. 먼저 모든 방법에 같은 Q-qualified 후보 풀을 고정한다. candidate generation 자체를 바꾸는 방법과 단순 integration scheduling을 한 표에서 직접 비교하면 원인이 섞인다. 첫 비교는 fixed-candidate experiment이고, 이후 정책에 따라 다음 후보까지 달라지는 연구는 continued-evolution experiment로 분리한다.

기준 정책은 naive joint, random replay, change-scope replay, frozen-evidence selection이다. naive는 추가 통합 검증 없이 정한 joint를 선택한다. random은 같은 D 재생 풀에서 미리 정한 난수 규칙으로 과제를 선택한다. change-scope는 수정한 단위·키·출처·모듈 경로와 계약 관련성을 고정 규칙으로 점수화한다. frozen-evidence는 사전에 정한 근거 순서를 유지한다. 적응 정책을 추가하더라도 같은 후보, 같은 D 정보와 같은 회계 규칙을 받는다.

정책이 할 수 있는 행동은 joint를 수용하기, 제한된 내부 adapter를 제안하기, 공개된 selector로 가지를 분리하기, 조상을 유지하고 보류하기다. 각 행동은 상태와 계약의 hash를 가진다. 분리 routing

을 선택했다면 그 routing 코드나 지침도 후보의 일부이고 selector가 틀릴 가능성을 평가해야 한다. 두 가지를 분리하면 자동으로 두 가지의 장점을 모두 얻는다고 가정하지 않는다.

계약 관련성 점수는 원인 확률의 인증이 아니다. 이름이 'units'라서 unit 과제를 먼저 돌리는 전략은 실용적인 fixed heuristic일 수 있다. 그 전략보다 복잡한 메타 컨텍스트 정책이 꼭 좋아야 한다는 전제는 없다. 최소한 frozen heuristic과 random, 가능한 경우 순차 재기반+재검증을 같은 예산에서 비교하고, 차이가 작으면 단순 방법으로 충분한 조건을 보고한다.

비용에는 모델 입력·출력 token, cached token, 도구 사용, CPU·wall time, 재시도, 진단·스케줄링·보수와 거절된 제안이 포함된다. 실제 돈을 계산하려면 실행 시점의 가격과 모델 정책도 필요하다. 현재 fixture의 한 단위가 실제 token·원·초·에너지로 변환되는 것은 아니다. 예산 축과 primary cap을 하나 선택한 뒤 다른 비용 지표는 sensitivity로 보고할 계획이다. 지금은 그 cap을 정하지 않았다.

공통 자료를 한 번 읽고 여러 검사를 하는 경우는 중복 비용을 피하되 원장을 남긴다. 교육용 가상 예로 자료 취득 2단위, 검증 각각 1단위의 두 검사를 같은 캐시로 수행하면 합계는 4단위다. 각 검사에 취득비를 다시 붙여 6단위로 쓰면 shared cost를 이중 계산한다. 반대로 한 정책만 무료 캐시를 받고 다른 정책은 2단위를 지불하면 비교가 불공정하다. 각 정책의 시작 캐시와 cache hit의 조건을 동일하게 고정해야 한다.

Q와 C/S의 최종 audit는 정책이 사용할 수 없는 연구 비용이다. 정책 예산에 숨은 정답 audit를 무상 포함시키지도 않고, 전체 연구 지출에서 완전히 빼지도 않는다. deployment/integration budget과 qualification/scoring overhead를 별도 합계로 보고한다. 후보 생성의 cost도 고정 풀 실험에서는 모두에게 공통으로 적고, 계속 진화 실험에서는 각 정책이 실제로 만든 이력의 비용으로 계산한다.

증거 cache는 버전별로 유효하다. task input, state, 후보, 계약, evaluator, 환경이 달라졌으면 과거 success는 현재 fresh success가 아니다. 예전 실패 궤적을 문맥으로 참고할 수는 있어도 현재 검증 완료의 점수를 줄 수 없다. 보수가 예전 실패를 고쳤을 가능성이 있는데 과거 negative lesson을 무조건 veto로 유지하면 잘못된 배제를 반복할 수 있다. 만료·갱신 규칙과 cached evidence 사용 범위를 먼저 고정한다.

좋은 integration policy의 주장에는 completion만 아니라 FH, R_excess, G 보존, no_gain, deferral, 전체 비용이 함께 필요하다. 모든 업데이트를 거절해 H만 지키거나 extra model calls로 더 많이 검사한 결과는 별도 tradeoff다. adaptive meta-policy의 필요성은 이 고정 후보 비교 이후에 독립적으로 시험한다. 현재 v3에는 learned meta-policy가 없고 고정 회계 fixture 정책만 있다.

17. 파일에서 정량 결과까지의 파이프라인과 증거 원장

현재 준비 단계와 앞으로 필요한 실행 단계를 한 줄로 연결하면 "원본 고정 → 공개 입력/정답 분리 → 후보 작성 → Q → 상태 패킷 → C → 정책 고정 → S → semantic 채점 → 그룹 분석"이다. 각 화살표가 파일 hash와 역할 계약을 남겨야 한다. 원본 공개 데이터의 filename만 기록하면 나중에 URL이 바뀌거나 라벨 수정이 발생했을 때 어느 버전을 썼는지 알 수 없다.

원본 inventory에는 출처 URL, acquisition receipt, 원본 bytes hash, license snapshot, 준비 코드 hash를 적는다. 후보 inventory에는 조상 hash, A/B 파일·참고자료 hash, 작성 입력·모델 메타데이터·정보 접근·시도 원장을 적는다. 모델 이름을 사람이 적었다는 사실과 그 모델을 실제로 호출했다는 attestation은 다르다. 현재 importer는 전자의 표기를 받아도 후자가 없으면 imported_unverified로 남긴다.

trial packet에는 실제 SKILL.md 내용과 hash, 활성화 순서, public input과 contract, task/source group, split role, repetition 배정, 모델·자원 설정이 들어간다. evaluator-only expected, gold patch, C 선정 판정이나 숨은 answer는 agent packet에 포함하지 않는다. 공개 contract의 SQL 함수 목록과 제한처럼 모든 arm이 알아야 할 harness 제약은 넣는다. 제한을 숨겨 유효한 답을 실패시키는 것은 정당한 독립 채점이 아니다.

v3의 prepare-trials는 실제 후보 파일을 읽어 120개 계획 패킷을 만들었다. 이는 24개 authored 과제에 S0/A/B/AB/BA를 배정한 준비 기록이다. 120개 모델 응답이나 120개 독립 현상 표본이 아니다. exporter가 skill 문자를 포함했다는 사실은 모델이 그 문자를 읽어 수행했다는 사실과 다르다. demo는 protected expected를 이용해 응답을 스크립트로 만들고 일부를 오염시키므로 과학적 모델 실행으로 재분류할 수 없다.

현재 fixture 기록은 8개 구성 사건, 192개 scripted response이다. 원장이 제안 실패·보류·no_gain을 보존하는지 확인하는 조작 자료다. fixture의 qualified나 witness라는 이름은 스크립트의 정해진 조건을 나타내며 실제 independently improved event와 다른 namespace로 취급한다. 이 수치를 자연 발생률이나 통합 정책의 우월성 성능표에 넣지 않는다.

실제 runner가 만들어야 할 최소 응답 행은 event/task/arm/order/repetition와 packet hash, 시작·종료 상태, 원시 출력, 도구 기록, 비용, 종료 이유다. 잘못된 JSON, timeout, 중단, 요청 전송 후 결과 미확인도 행을 남긴다. 응답이 정상인 행만 가져오면 completion과 cost가 모두 낙관적으로 바뀐다. 알려진 external outage와 agent-caused timeout은 같은 원인으로 합치지 않고, 재시도 정책을 실행 전 고정한다.

import는 원시 응답을 보존한 채 계약에 맞게 재채점한다. task ID와 state/version, packet·parent link, model declaration, 출력 경로가 서로 맞는지 검사한다. 데이터 출처와 실행을 인증할 수 없다면 계약 통과는 '수입된 미검증 출력의 계약 통과'일 뿐이다. 실제 능력, 독립 작성, 자연 발생, 모델 회복을 인증하는 문구로 올리지 않는다. 파일 hash는 작성자의 권한이나 실행 주체를 증명하지 않는다.

verify는 단순히 모든 파일의 hash가 manifest와 같다고 끝나면 부족하다. 원시 응답에서 score를 다시 계산하고, 원장으로 비용과 분모를 다시 집계하며, 선택 상태와 summary가 일치하는지 비교해야 한다. 누군가 score와 manifest hash를 함께 바꾸어도 원시 응답 재채점이 다르면 거절해야 한다. 마찬가지로 state·provenance를 함께 수정해 imported_unverified를 real_model로 바꾸는 변경도 거절해야 한다.

로컬 JSON 분리는 독립 evaluator 서비스가 아니다. 현재 fixture 프로세스는 정답 파일에 접근할 수 있다. 따라서 '정답을 모델 packet에서 제외했다'는 검사와 OS·서비스 수준으로 agent 접근을 차단했다는 검증을 나눈다. 현재 후자는 구현 완료로 쓰지 않는다. actual benchmark 전에 isolated mount, 숨은 테스트 접근, 경로 우회와 cache 잔존을 검토하는 계획이다.

기존 코드·실행을 새 버전으로 덮어쓰지 않는다. grader가 바뀌면 새 output directory와 새 source hash로 run을 만들고, 원래 packet/run의 의미는 그대로 남긴다. v5는 구현 코드의 새 실행 프로젝트가 아니라 원고·해설서·방법 계약·출처 ledger의 새 버전이다. 이 작업에서 native benchmark code, 모델, 원본 ZIP 코드와 active skill을 실행하거나 바꾸지 않았다.

18. 분석, 가중치, 반증과 자연 발생률

분석의 순서는 고정한 주 추정량 θ부터다. Q-qualified 후보 쌍을 정하고 사전 배정된 S 입력 전체의 paired d를 계산한다. 다른 benchmark의 다른 성공 정의를 한 평균으로 합치지 않는다. SkillsBench native completion이 main이며 SciFact의 schema-valid label accuracy는 준비용 adapted pilot이다. 둘은 같은 평가 함수의 두 복제 실험이 아니다. SWE-bench external도 native 기능 테스트 결과로 별도 보고한다.

가중치의 영향은 직접 계산할 수 있다. 다음 수치는 교육용 가상 예다. family 1은 100과제의 d 평균이 −.10, family 2는 2과제의 d 평균이 +.50이다. 문제별 동일 가중 평균은 (−10+1)/102≈−.0882이고 family별 동일 가중 평균은 (−.10+.50)/2=.20이다. 효과의 부호도 달라진다. 마음에 드는 평균을 고르는 대신 어떤 source 분포로 일반화하려는지 정해 사전에 가중치를 잠근다.

primary θ가 음수여도 모든 joint가 무너진 것은 아니다. 일부는 좋아지고 일부는 나빠져 평균이 음수가 될 수 있다. strict1110는 특정 기존 완료 과제에서만 계산하며, 별도로 신규 능력 손실과 order sensitivity를 본다. event-level "어느 과제든 witness가 하나 있는 비율"은 task-level 비율과 다르다. 한 event에 100과제를 배정하면 5과제만 배정할 때보다 '하나 이상 발견'될 기회가 커지므로 과제 배정 수를 동일하게 하거나 검출 기회의 차이를 명시해야 한다.

교육용 가상 assigned 사건-과제 100개에 confirmed witness 3개와 unresolved 2개가 있으면, unresolved가 전부 비witness일 때의 하한은 3%, 전부 witness일 때의 상한은 5%다. 남은 98개만 분모로 써 3/98을 대표 빈도로 바꾸지 않는다. 이 범위도 Q-eligible·고정 생성기·정해진 과제 배정에 조건부다. 전체 후보 제안에서의 자격 통과율과 자연 충돌 빈도를 따로 제시한다.

0개 witness도 유효한 결과다. 그 자체가 효과가 절대 없다는 증명은 아니다. 명시적으로 독립 Bernoulli 표본이라는 단순 가정을 둔 교육용 계산에서, n번 모두 0일 확률은 (1−π)^n이다. 이를 .05와 같게 놓은 상한은 1−.05^(1/n)이다. n=100이면 약2.95%, n=20이면 약13.91%다. 실제 데이터가 family/history로 의존하면 실행 행 n을 여기에 그대로 넣을 수 없다. 이 계산은 정밀도와 독립 단위를 왜 구분해야 하는지 보여 준다.

secondary 분석은 탐색과 확증을 구분한다. 여러 순서·모델·family·문턱 가운데 큰 효과만 제시하면 다중 선택이 된다. 한 primary θ, secondary family와 correction 규칙, effect threshold와 stopping rule을 고정한다. Holm 같은 방법을 제안할 수 있으나 현재 수행된 유의성 검정이나 보정 결과는 없다. pilot의 흔들림을 본 뒤 S 결과에 맞춰 문턱을 바꾸는 것도 피한다.

결측은 원인별로 남긴다. agent가 output contract를 어기거나 허용 시간 안에 끝내지 못한 것은 완료 실패다. 외부 서버 장애로 아예 응답 관측이 없는 것은 censoring 또는 unresolved로 기록한다.

predeclared retry, out-of-budget 발생, 호출이 전송됐지만 결과를 잃은 경우도 비용 원장에 보존한다. complete-case 평균을 대표 결과로 쓰지 않고 전체 배정 기준의 실패율·결측률·보수적 범위를 함께 보고한다.

계약 원인의 반증은 대안 설명과 연결된다. 길이를 맞춘 무관련 블록도 같은 손실을 재현하면 특정 의미 충돌 설명은 약해진다. AB/BA의 한 순서만 영향을 받으면 순서 조건부 설명으로 좁힌다. 동일 자원을 제공하면 차이가 사라지거나 valid alternative grader가 실패를 고치면, 최초의 '내용 충돌' 주장을 수정해야 한다. scoped adapter 개입이 개선하지 못하면 그 boundary가 주된 원인이라는 가설도 약해진다.

정책 우월성의 반증은 더 명확하다. 같은 총비용에서 random/change-scope/frozen 기준과 실용적 차이가 없거나, 새 G 능력을 버려 H 보존만 높였다면 전체 integration 이득이라고 주장하지 않는다. 가장 단순한 fixed heuristic과 동등한 결과도 연구의 유효한 결론이다. 자연 후보에서 strict witness가 희귀하면 crafted stress를 더 만들어 빈도를 높였다는 결론으로 바꾸지 않는다.

공개 데이터 노출, incomplete verifier, 후보 생성 모델의 제한, pairwise 범위도 결론을 좁힌다. 모든 두 가지 조합이 성공한다고 세 가지 이상 조합도 성공하는 것은 아니다. 고차 interaction에는 별도 처리 조건·예산·분모가 필요하다. 첫 주의 pair 연구를 전체 스킬 생태계에 대한 보장으로 확대하지 않는다.

19. 첫 주 연구 이행 계획과 아직 잠그지 못한 것

첫 주의 완료 조건은 "논문에서 필요한 실험을 이미 했다"가 아니라, 실행 전의 계약과 오류 경계를 리뷰 가능한 형태로 고정하는 것이다. 4페이지 원고는 이를 연구 설계로 명시한다. 다음 목록은 day 단위 작업 제안이며 지금 수행했다는 체크리스트가 아니다. 모델 ID와 입력 전송 범위, 요청·token·비용 한도가 확정되지 않아 actual trial을 시작했다는 표시는 없다.

첫날에는 source inventory, 원본 bytes와 asset terms, 공통 조상·event schema를 고정한다. SkillsBench의 두 commit 역할, SciFact source hash와 component split, SWE-bench native base/test 경계를 각각 확인한다. 실제로 받은 asset의 hash가 없는 항목은 metadata-ready로만 둔다. 전체 코드 라이선스가 개별 data terms의 확인을 대신하지 않는다.

둘째 날에는 grader를 먼저 공격적으로 검토한다. 유효한 대안 질의·더 넓은 인용·정상 유형·경계 입력이 false failure를 만들지 않는지 확인하고, 잘못된 source·숫자·condition·ID를 넣은 답은 실패하는지 시험한다. 정답 복제 테스트만 통과해도 독립 채점이 충분하다고 결론내리지 않는다. 원래 기준과 바뀐 기준을 버전으로 남긴다.

셋째 날에는 task split과 Q/C/S/D 권한, clean reset, agent packet, tokenizer와 resource matching을 설계한다. v3의 Python 표준 라이브러리 prototype이 작동했다는 사실만으로 SkillsBench native 환경이 준비되지는 않는다. 검토한 native 의존성 문서는 Python≥3.12와 BenchFlow 계열을 요구하지만 현재 실행·설치를 수행하지 않았다. runner를 새 의존성 없이 준비했다고 말하지 않는다.

넷째 날에는 작은 pilot에서 branch qualification yield와 paired 확률 추정의 정밀도를 볼 계획이다. pilot 입력은 main의 최종 S와 구분한다. 개별 가지가 실제로 개선되는지, C에서 H/G가 비어 버리는지, forced와 availability의 노출이 충분한지 먼저 확인한다. Q 문턱을 통과하는 후보가 없다는 결과도 보존한다. 현 단계는 이 pilot 실행 전이다.

다섯째 날에는 primary endpoint, 가족·이력 가중치, canonical order, qualification margin, high/low witness gates, 반복 수, 비용 cap, exclusion/retry와 multiplicity를 잠근다. pilot-derived proposal과 preregistered final rule을 구분한다. 규모가 부족하면 정밀도가 낮은 pilot 연구로 범위를 줄이고, 자신 없는 population claim을 주장하지 않는다.

여섯째 날에는 ledger·packet·import·semantic verify의 끝 연결을 감사한다. hash만 다시 쓴 조작이 score 재계산을 통과하는지, 다른 task·모델 이름을 붙인 미검증 응답이 real_model로 올라가는지, invalid/중단 원장이 남는지 본다. 연구 프로토콜의 논리 검사와 실제 실행 주체의 인증을 나눠 보고한다.

일곱째 날에는 4페이지 설계 원고와 장문 해설서, 방법 계약, source ledger를 함께 검토한다. 준비 완료·미완료·unknown을 공개하고 원고와 실행 artifact의 주장을 맞춘다. 실제 pilot을 진행했다면 그 작은 관측과 구성 fixture를 separate report로 보관한다. 지금 완성하는 v5는 이 단계의 설계 문서이고 논문 제출·운영 배포·active skill promotion을 포함하지 않는다.

현재 특히 중요한 미정 항목은 실제 모델 snapshot과 tokenizer, source-family 인벤토리, 서로 격리된 작성·audit 환경, Q improvement gate, 효과와 precision 목표, final 비용 한도, native SkillsBench

adapter, SWE-bench 환경·asset terms다. 이 항목들은 파일을 많이 만든다고 사라지지 않는다. locked=false를 유지한 이유이며, NR과 unknown은 연구 진행을 설명하는 정보다.

20. 자주 나오는 질문과 기술적 답변

Q1. 결국 SKILL.md를 여러 개 붙이면 성능이 떨어진다는 논문인가? 연구 대상은 개별 개선을 별도로 통과한 동일 조상 업데이트의 조합이다. 단순한 많은 지침의 주입 손실과 겹치는 부분은 있지만, Q-conditioned estimand와 생성·자격·C/S 분리가 핵심이다. 같은 결과가 길이 대조에서도 보이면 특정 내용 충돌이 아닌 길이 조건으로 설명을 좁힌다.

Q2. A와 B가 각각 좋아졌는데 baseline이 t0를 이미 푸는 것은 모순인가? 개선은 Q 분포의 평균과 정해진 보존 규칙으로 확인한다. retained t0에서는 원래도 잘 풀 수 있다. 모형에서는 A가 tA의 파싱을, B가 tB의 ID를 고쳐 각각 성공 개수를 1에서 2로 늘린다. t0의 joint 실패는 그 획득을 합치는 과정의 별도 패턴이다.

Q3. 세 입력 표가 실제 독립 작성과 Q 자격을 증명하는가? 증명하지 않는다. 함수 정의가 논리적 implication의 반례를 만들 뿐이다. actual candidate에는 따로 격리된 작성 이력, frozen Q와 S가 필요하다. 같은 수학 모형에서 Q 평균을 계산했다고 실제 현상 표본을 만들었다고 세지 않는다.

Q4. PB가 ID를 어떻게 아는가? R0와 RA가 공개 입력의 record_id를 중간 artifact에 넣는다. PB는 허용하는 Won artifact의 ID를 복사한다. 모든 arm의 공개 입력과 권한은 같다. PB가 evaluator-only 정답을 보거나 B arm만 추가 정보를 받는 설정은 아니다.

Q5. 타입이 안 맞아 거절하면 틀린 답을 내는 것보다 안전하지 않은가? 그럴 수 있다. 이 모형은 거절을 조용한 오답과 분리한다. 다만 정해진 completion 지표에서는 요구한 과제를 끝내지 못했으므로 실패다. 정책 평가에서는 deferral과 비용, G 상실을 따로 보여 주어 거절의 tradeoff를 해석한다.

Q6. 왜 S에서 더 좋은 단일군을 baseline으로 고르지 않는가? 표본 잡음으로 높아진 군을 골라 비교하면 조합 손실을 과장할 수 있다. primary는 두 단일군의 평균이라는 미리 정한 기준이다. 강한 deployment baseline은 D/Q의 잠긴 규칙으로 선택하고 independent S에서 평가하는 secondary 정책으로 둘 수 있다.

Q7. 음수 τ는 왜 strict witness가 아닌가? (.80,.95,.95,1.00)에서는 joint가 최고지만 τ=−.10이다. 천장과 비가산성이 작용한다. strict1110는 조상·A·B의 높은 성공과 joint의 낮은 성공을 직접 요구하며, 불확실한 사례는 unresolved로 남긴다. 두 수치를 하나의 '충돌 점수'로 합치지 않는다.

Q8. 파일로 gold를 분리했으면 독립 평가기인가? 모델 packet에서 답을 제외했다는 점과 agent 프로세스가 그 파일을 읽지 못한다는 점은 다르다. 현재 demo는 protected expected를 직접 읽어 scripted response를 만든다. actual run에는 별도 mount·서비스·접근 기록 검증이 필요하며 완료했다고 표시하지 않는다.

Q9. import한 결과에 모델 이름이 있으면 실측으로 써도 되는가? declared metadata만으로는 아니다. source와 parent packet, configuration, 원시 응답·cost·실행 증명을 검토해야 한다. 현재 importer는 그런 외부 인증이 없으면 imported_unverified로 보존한다. 계약 재채점의 유효성은 모델 실행의 인증과 별도다.

Q10. 531개 SciFact가 이미 있으니 효과 논문을 바로 쓸 수 있는가? 데이터와 라벨 준비는 효과 실험의 한 부분이다. 자연 후보 생성, 개별 개선 Q, C/S 실행과 정보 격리가 아직 없다. 또한 이 test는 public original-train adaptation이다. 공개 데이터 기반 pilot이라는 조건 없이 native 공식 성능이나 미노출 일반화로 표현할 수 없다.

Q11. 44개 label mutation은 SkillMerge 조합 실패의 정답인가? 아니다. train-only evaluator-stress 파일이며 원본 annotation을 수정하지 않았다. SkillMerge main은 정정된 정상 평가 기준으로 수행하고, 평가 오류를 사용한 constructed track은 별도로 표시한다. 이 수를 자연 후보 이력이나 실제 회복 사건 수로 세지 않는다.

Q12. 많은 반복 실행이면 통계적으로 강해지는가? 한 셀의 성공확률 precision은 좋아질 수 있다. 하지만 같은 source family와 candidate history만 반복하면 새로운 현상 분포의 표본 수는 늘지 않는다. crossed dependence, qualification yield, 실제 independent cluster 수를 반영해 power를 정한다.

Q13. 모든 업데이트를 거절하는 방법이 최선인 경우도 있는가? H 보존에 중요할 수 있으나 G 신규 능력과 completion 기회를 잃는다. 같은 비용에서 no_gain·deferral·FH·R_excess·G retention을 함께 보고한다. 업데이트 없는 reference는 필요하지만 그 상태 유지가 전체 연구 목표의 개선을 자동으로 뜻하지 않는다.

Q14. 계약 정리가 LLM의 안전성을 보장하는가? 정리는 정의한 deterministic 함수·의미 전제·reset 가정 안에서만 성립한다. 실제 모델이 태그와 조건을 일관되게 보존하는지는 측정해야 한다. 안전한 함수 연결과 확률적 prompt 해석, 정적 AST 검사와 native 기능 테스트는 서로 다른 근거다.

Q15. 첫 주 끝에 독자가 얻는 확실한 결과는 무엇인가? 검증 가능한 질문, 기준 지표, 세 입력 반례와 조건부 보존 논리, concrete task/control 설계, 현재 corpus·software 준비 근거와 미완료 경계를 얻는다. 실제 effect size, 자연 prevalence, 정책 우월성은 아직 NR이다. 다음 실험은 이 설계를 사전 고정한 뒤 별도의 원시 실행 증거를 쌓는 단계다.

본문의 [1]–[8]은 4페이지 원고의 동일 참고문헌에 대응한다. 공식 URL·읽은 범위·주장 연결·로컬 원본 hash는 동봉한 SOURCES.json에 있다. 수학 모형, 가상 예, 직접 준비 기록, 외부 저자의 보고를 섞지 않고 검토할 수 있도록 방법 계약도 별도 JSON으로 제공한다.